

Nexus: Depth-Adaptive KV-Cache Splicing and Retrieval-Decoupled Tool Routing for Agentic LLMs on Unified Memory

Mustafa Arslan
Independent Researcher
Istanbul, Türkiye
mustafarslan35@gmail.com

Abstract—Agentic large language models (LLMs) on the Model Context Protocol (MCP) re-encode verbose tool schemas every turn, so prefill—quadratic in sequence length—dominates time-to-first-token (TTFT) as the tool registry grows. Nexus’s primary lever is to *decouple routing from the schema-prefill cost*: an INT8 semantic lookaside buffer (SLB) with a calibrated cross-encoder margin gate selects tools by retrieval, and arguments are generated over a compressed textual signature (median 19 tokens) rather than over spliced key/value (KV) cache. This path is depth-independent: routing accuracy stays near 89% as the registry scales to 250 tools—where a concatenate-all-schemas baseline overflows the context window entirely—and it reaches a first-argument token $1.66\times$ sooner than a full-schema re-prefill at an $\approx 80\%$ main-context token saving. As a secondary, bounded lever we transplant a compiled schema KV block directly into the live context. This is fundamentally limited by rotary position embedding (RoPE) phase drift: an anchored splice is output-exact, but off-anchor placement corrupts attention, so beyond a threshold $P = 256$ Nexus repairs the seam with a depth-adaptive suffix redecode that escalates to a full re-prefill. The resulting never-regress property is a guarantee on output *fidelity* (top-1 agreement, $D_{KL} \approx 0$)—not on latency, which can dip to $0.98\times$ before converging to parity—alongside a $1.1\text{--}1.7\times$ TTFT speedup at moderate depth that narrows to parity at deep context. Two negative results bound the design: the off-anchor RoPE fidelity boundary, and the failure of a reference-free drift gate to predict drift (Spearman $\rho = 0.193$). All measurements are from one model tuple (Qwen2.5-14B-Instruct Q4_K_M) on Apple-silicon unified memory; the *qualitative* boundaries generalize, while the *quantitative* envelope is tuple-specific.

Index Terms—large language models, tool use, KV cache, rotary position embedding, agentic systems, retrieval, inference systems

I. INTRODUCTION

Tool-augmented LLM agents increasingly standardize on the Model Context Protocol (MCP) [1], presenting the model with a registry of callable tools, each described by a verbose JSON schema. As the registry scales to hundreds of tools, these schemas dominate the prompt. Because self-attention is $\mathcal{O}(N^2)$ in sequence length N [2], re-encoding tool schemas every turn imposes a prefill wall that scales poorly with the number and verbosity of available tools.

An appealing optimization is to compile each schema once into a KV block and transplant it into the live context at inference time, paying the prefill cost only once. We find this is fundamentally constrained by the position dependence of rotary position embeddings (RoPE) [3]: a block compiled for

one absolute position cannot be relocated to an arbitrary depth without corrupting attention. Prior serving systems sidestep the problem by never moving cache (paged and prefix-shared KV [4], [5] keep blocks at the positions where they were computed). Nexus instead asks how far a *relocated* schema block can be trusted, and what it costs to repair it.

This paper presents the design, implementation, and measured envelope of *Nexus*, a user-space KV-cache serving prototype for agentic tool use on Apple-silicon unified memory (UMA). Every mechanism below is anchored to a concrete symbol in the codebase, and every number to a committed artifact under `results/v2.0_canonical/`. We make four contributions, tagged by evidence class:

- 1) **Retrieval-decoupled routing** (*measured*), the more durable and transferable result. Tool selection runs over an INT8 semantic lookaside buffer (SLB) with a calibrated cross-encoder gate, and arguments are generated over a compressed textual signature, avoiding the splice seam. It is depth-independent, scales to 250 tools where the concatenate-all baseline cannot run, and delivers a $1.66\times$ first-argument latency win at $\approx 80\%$ token saving (§V, §VII).
- 2) **Depth-adaptive recompute with a never-regress guarantee** (*measured*), a bounded, characterized secondary lever. Past a 256-token splice threshold, Nexus re-decodes a suffix fraction $R(n_{\text{past}})$ that scales with depth to 100%, holding output fidelity exact and converging gracefully to prefill parity (§IV, §VII).
- 3) **Systems mechanisms for UMA** (*implemented*): transposed-V splicing for soft-capped attention models and cache-line hardening of the block allocator (§VI).
- 4) **Two bounding negative results** (*well-supported*): the off-anchor RoPE fidelity boundary and the failure of reference-free drift gating (§IV).

We are explicit about scope. All benchmarks were executed on an Apple M4 Max SoC (16-core CPU, 40-core GPU, 16-core Neural Engine) equipped with 64 GB Unified Memory and a 1 TB NVMe SSD, running macOS/Darwin 25.5.0 (arm64), at git 1ce4aa4 / llama.cpp cb2463bb. All measurements use Qwen2.5-14B-Instruct Q4_K_M with nomic-embed-text-v1.5. Artifacts are un-

der results/v2.0_canonical/. The transposed-V splice mechanism is additionally implemented and fidelity-tested on a second, soft-capped model (Gemma-2-9B). However, the physical splicing mechanism is strictly limited to local execution of flash-attention-compatible, non-soft-capped GGUF architectures. Splicing cannot execute under local Gemma4 execution (due to soft-capping forcing a transposed layout that the splicer rejects, and loading errors under the pinned llama.cpp version) or cloud execution (due to the physical nature of cache transplantation requiring direct memory copies into local physical cache addresses). Soft-capped and cloud models must fall back entirely to text-prefill (Path B, yielding a $1.0\times$ speedup). Sample sizes are small ($n \leq 30$ for end-to-end arms), and the deep splice is validated at the fidelity-and-latency level, not as a multi-turn production path. We view the negative results and the measured envelope as the durable contribution.

Three research questions organize the paper. **RQ1**: how far can a relocated schema block be trusted before its next-token distribution departs from a full prefill (§IV)? **RQ2**: what does repairing that drift cost in TTFT (§VII)? **RQ3**: can routing and argument generation avoid the splice path entirely (§V, §VII)?

Artifact availability. Every reported number traces to a committed artifact under results/v2.0_canonical/ at git 1ce4aa4 (llama.cpp cb2463bb), for the model tuple Qwen2.5-14B-Instruct Q4_K_M with nomic-embed-text-v1.5. The artifact bundle is available from the author on request pending release; a public archive with a DOI will be added on publication.

II. RELATED WORK

Tool retrieval and schema compression. The two standard responses to schema bloat both act before the model. Retrieval-augmented tool selection—exemplified by RAG-MCP [6]—fetches only the relevant MCP schemas prior to prefill. Schema-level compilation [7] and learned latent tool retrieval [8] shrink or replace tool descriptions at the application and embedding levels, the latter requiring trained alignment. Nexus adopts the same retrieval principle to decouple routing but adds a complementary system-level lever: direct KV transplantation for the schemas that remain, with a zero-shot calibrated margin gate over frozen embeddings. We position Nexus against RAG-MCP and in-place KV-serving systems in Table I; because those numbers are reported on different hardware and workloads, we treat them as contextual rather than a controlled head-to-head, which we leave to future work.

KV-cache management for serving. Paged and prefix-shared KV management [4], [5] and dynamic virtual-memory schemes [9], [10] eliminate fragmentation and reuse cache *in place*; they do not relocate a block to a new position, which is precisely the regime Nexus characterizes. RedKnot [11] pushes reuse to the granularity of individual attention heads but keeps cache in place, whereas Nexus relocates coarse per-tool blocks. Fail-closed lowering of resident KV claims [12] formalizes when a runtime may treat a cached-KV obligation as satisfied; Nexus’s splicer takes the same fail-closed stance, declining transposed-V splices on discrete-GPU back-ends rather than

risk corruption. Prefill/decode disaggregation and chunked prefill [13]–[15] attack the same prefill cost from the scheduling side and are complementary. Transactional speculative KV [16] similarly separates stable from transient KV state.

RoPE, attention kernels, and decoding. Context-window extension via frequency interpolation [17] shares RoPE machinery with our reanchor path but targets longer contexts, not cache transplantation. FlashAttention [18], [19] determines the V-cache layout that dictates our transposed-V handling. Constrained decoding [20]–[22] underlies the FSM/JSON masking used for argument generation. Nexus targets a different regime—relocating compiled schema blocks under unified memory—and we make no outperformance claim against these systems. The SLB scan mechanism and .atb page alignment build directly on the neuro-symbolic memory substrate (Memory Palace/Atlas index, Trace DAG, SLB) introduced in Aeon v3 [23], extending them to direct, zero-copy physical cache transplantation under UMA.

III. BACKGROUND AND MOTIVATION

A. Self-Attention and the Schema-Prefill Wall

For a sequence of length N and per-head dimension d , scaled dot-product attention computes $\text{softmax}(QK^\top/\sqrt{d})V$ with $Q, K, V \in \mathbb{R}^{N \times d}$, incurring $\mathcal{O}(N^2d)$ time and $\mathcal{O}(N^2)$ attention memory per layer [2]. This expression is definitional, not a new bound; we cite it only to locate the dominant cost. In agentic serving, $N = H + \sum_k S_k$ where H is the conversation history and S_k the token length of the k -th tool schema. Because the schemas are re-encoded every turn, reducing their prefill contribution is the dominant lever, and the two standard baselines—concatenate all schemas, or retrieve K and prefill those—both pay this cost online.

B. RoPE Phase Drift

RoPE injects position by rotating query/key feature pairs by a position-dependent angle [3]. For feature pair i with base frequency $\theta_i = b^{-2i/d}$ (the runtime model uses base $b = 10^6$), a key compiled assuming anchor position m_0 but consumed at position n_{past} accrues a phase error

$$\Delta\theta_i = (n_{\text{past}} - m_0) \theta_i. \quad (1)$$

This is a definitional consequence of RoPE, not an empirical predictor of the divergence values in §IV. Because attention scores depend on the *relative* rotation between query and key, a transplanted block whose keys carry the wrong absolute phase is read off-axis by every subsequent query. The error grows with the placement offset ($n_{\text{past}} - m_0$) and is non-uniform across dimensions (high-frequency pairs drift fastest), so a pre-compiled schema block is faithful only in a neighborhood of its compile anchor. The codebase makes this boundary concrete: the splicer computes `delta_pos = n_past - header->base_pos` (`nexus_kv_splicer.cpp`), and the orchestrator declines a bare splice once n_{past} exceeds `MAX_SPLICE_POS = 256` (`nexus_orchestrator.cpp`).

TABLE I

NEXUS IN CONTEXT. IN-PLACE KV-SERVING SYSTEMS NEVER RELOCATE A COMPILED BLOCK; NEXUS RELOCATES PER-TOOL BLOCKS AND DECOUPLES ROUTING FROM THE SPLICE PATH. NON-NEXUS NUMBERS ARE REPORTED ON DIFFERENT HARDWARE AND WORKLOADS ($\dagger$: CONTEXTUAL, *not* A CONTROLLED HEAD-TO-HEAD; SEE RESULTS/COMPARISON_SOURCES.MD).

System	KV strategy	Routing	Reported metric (native HW/workload)	Workload / HW	H2H?
RAG-MCP [6]	in-place (prompt-level)	retrieve- K + prefill	43.1% vs 13.6% tool selection; $>50\%$ prompt tokens †	MCP stress test, LLM eval	no
vLLM / PagedAttn [4]	in-place (paged)	—	2–4 $\times$ throughput †	ShareGPT, A100/A10G	no
SGLang / RadixAttn [5]	in-place (prefix reuse)	—	up to 6.4 $\times$ throughput †	LLM programs, A10G/A100	no
vAttention [9]	in-place (virtual contig.)	—	up to 1.23 $\times$ vs paged kernels †	GPU serving	no
RedKnot [11]	in-place (head-aware)	—	resource-efficiency gains (qualitative) †	GPU serving	no
Nexus (this work)	relocate (per-tool)	SLB + margin gate	routing 89%@250 tools; first-arg 1.66 $\times$; deep-splice 1.1–1.7 $\times$	GitHub-MCP, M4 Max UMA, Qwen2.5-14B	—

C. Depth-Adaptive Recompute

Rather than treat 256 as a hard wall, Nexus repairs drift by re-decoding the trailing tokens of the spliced schema, with a fraction that grows with depth. Let $M = 256$ be the splice threshold (`max_splice_pos_`), $R_{\text{base}} = 5\%$ the baseline recompute fraction (`recompute_pct_`), and K the multiple of M at which recompute reaches 100% (`recompute_full_mult_`). The effective fraction is

$$R(n_{\text{past}}) = \begin{cases} R_{\text{base}}, & n_{\text{past}} \leq M, \\ R_{\text{base}} + \frac{n_{\text{past}} - M}{M(K-1)}(100 - R_{\text{base}}), & n_{\text{past}} > M, \end{cases} \quad (2)$$

clamped to 100%. This matches `eff_pct()` in the code exactly. At deep positions the fraction approaches 100%, which is numerically a full text prefill ($D_{\text{KL}}(\mathbf{p}_0 \parallel \mathbf{p}_{\text{splice}}) = 0$); K is the knob that trades TTFT flatness for how early full recompute engages.

IV. PHYSICAL LIMITATIONS AND NEGATIVE RESULTS

We present the negative results first because they motivate the architecture: they fix the shape of the repair curve and force routing to be depth-independent rather than to depend on a deep splice.

A. The Anchored/Off-Anchor Fidelity Boundary

We evaluate splice fidelity by the divergence of the model’s next-token distribution from a recompute-equivalent reference. At the compile anchor ($\Delta\text{pos} = 0$) an anchored splice is output-exact: $D_{\text{KL}}(\mathbf{p}_0 \parallel \mathbf{p}_{\text{splice}}) \approx 0$ with top-1 agreement = 1.0. Once the block is placed off-anchor, the phase error of (1) perturbs the distribution. Fig. 1 traces this perturbation directly: with recompute *disabled* (a bare splice, so the curve shows the raw off-anchor drift the repair must overcome), we sweep the placement offset from the anchor to $\Delta\text{pos} = 2048$ and measure the next-token divergence against a full-prefill reference at each depth (10 queries/offset, `dkl_sweep.json`). Two facts stand out. First, the RoPE reanchor is effective: the divergence rises from its ≈ 0 anchor floor to $\sim 10^{-2}$ nats within the first 64 tokens and then *plateaus*—it stays in the 0.008–0.038-nat band, with top-1 agreement = 1.0, across the entire 0–2048 range, so a contiguous-suffix bare splice never fails catastrophically at these depths. The threshold $P = 256$ is therefore a *conservative* repair-start—where the drift has clearly separated from the

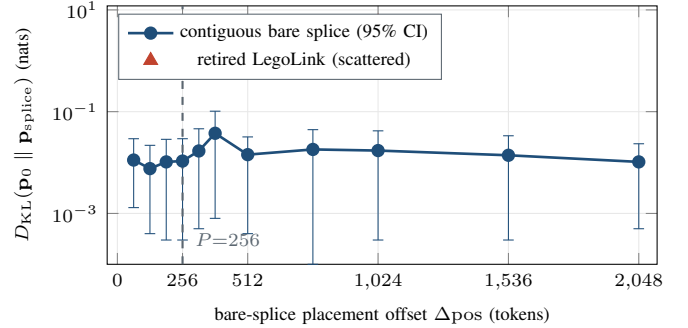


Fig. 1. Bare-splice next-token divergence vs. placement offset, recompute disabled (`dkl_sweep.json`; 10 queries/offset, 95% bootstrap CI). The contiguous-suffix splice holds $D_{\text{KL}} \sim 10^{-2}$ nats with top-1 agreement = 1.0 across the whole 0–2048 range—two orders of magnitude below the retired LegoLink scattered-recompute point (5.7 nats at $\Delta\text{pos}=1024$). The enforced threshold $P=256$ (dashed) is a conservative repair-start, not a fidelity cliff.

anchor floor while top-1 is still safe—rather than a sharp fidelity cliff.

Second, the retired scattered partial-recompute configuration (LegoLink) tells the complementary cautionary story: at $n_{\text{past}} = 1024$ it left the divergence as high as $D_{\text{KL}} \approx 5.7$ nats (`rope_boundary_gate.json`)—two orders of magnitude *above* the contiguous-suffix bare splice at the same depth (Fig. 1). Scattered recompute thus actively corrupted the distribution rather than repairing it, which is precisely why the current design repairs a *contiguous* trailing suffix and why that mechanism is retired. The production splicer still blocks bare splices beyond $n_{\text{past}} = 256$ and escalates the repair fraction of (2) with depth: the bare drift is small but nonzero, and the depth-adaptive redecode drives it to exactly 0 (Table III), trading a little TTFT for a hard never-regress fidelity guarantee rather than merely a small residual.

B. The Failure of Reference-Free Drift Gating

A natural optimization for the Path-A (≤ 256) to Path-B (> 256) transition is a cheap, reference-free runtime proxy—per-head preceding-context K-variance—that predicts when a deep splice will drift catastrophically. Profiling this proxy against the true per-head drift on the 14B model (`gating_nogo.json`) rules the approach out. The proxy does not predict drift: mean Spearman $\rho = 0.193$, well below the 0.40 target. Per-head

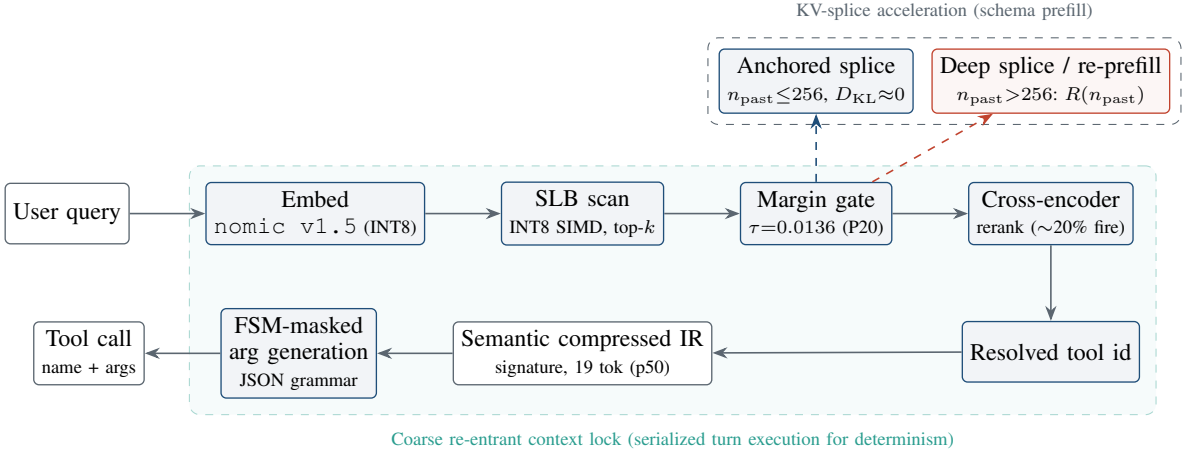


Fig. 2. Nexus request flow. Routing (blue, lower path) depends only on the query embedding and the tool registry, never on the deep context, so it is depth-independent. KV-cache acceleration (upper, dashed) is a separate concern: an anchored splice is output-exact for $n_{\text{past}} \leq 256$, while beyond it the depth-adaptive repair $R(n_{\text{past}})$ of (2) escalates to a full re-primfill. Arguments are generated over a compressed textual signature, not over spliced KV.

TABLE II

REFERENCE-FREE DRIFT GATE: PER-HEAD DRIFT VARIES (MAX 175–207, MEAN 91–105), BUT THE K-VARIANCE PROXY FAILS TO RANK-CORRELATE WITH IT ($\rho = 0.193$) (GATING_NOGO.JSON).

n_{past}	ctx	max drift	mean drift	Spearman ρ
256	on	207.1	105.2	0.147
256	off	195.3	97.8	0.221
1024	on	193.2	98.4	0.182
1024	off	189.1	96.4	0.250
2048	on	194.7	97.6	0.159
2048	off	175.4	91.0	0.197
mean Spearman ρ (target ≥ 0.40)				0.193

drift is not itself absent—max drift ranges 175–207 and mean drift 91–105 across depths and both on- and off-topic contexts (Table II)—but the K-variance proxy fails to rank-correlate with that variation, so a scalar-threshold gate driven by it cannot separate high- from low-drift heads and would fire indiscriminately. An effective online gate would require a true reference (the full prefill cost we seek to avoid) or a custom attention kernel. This justifies keeping the deterministic depth-adaptive curve of (2) as the repair mechanism.

V. ARCHITECTURE: RETRIEVAL-DECOUPLED ROUTING

The negative results above dictate the architecture (Fig. 2): routing must be depth-independent rather than depend on a deep splice. Arguments, in turn, are generated over a compact textual signature rather than over spliced schema KV, which both keeps the main-context prompt small and avoids relying on a splice inside the drift-prone regime of §IV.

A. Semantic Lookaside Buffer and Gate

Tool selection runs over a dense SLB (`nexus_slb.cpp`): tool embeddings from `nomic-embed-text-v1.5` are INT8-quantized and scanned with a branchless SIMD dot-product (NEON / AVX-512-VNNI / AVX2). The embedding of a tool

concatenates its name and description under the asymmetric search_document: prefix, mirrored by search_query: on the query side; the tool name carries dominant routing signal. The top candidates pass a margin gate: if the top-two score margin exceeds a calibrated threshold the decision auto-routes, otherwise it escalates to a fine-tuned MiniLM-class cross-encoder. The threshold is calibrated to the 20th percentile of adversarial-pair margins, $\tau = 0.0136$ (`calibration.json`), so the cross-encoder fires on the lowest-confidence $\sim 20\%$ of decisions. Retrieval depends only on the query embedding and the registry, never on the deep context, so it is inherently depth-independent.

B. Execution Sidecar and Compressed IR

When a candidate splice is required to validate routing in a fresh sequence, Nexus allocates a temporary *sidecar* sequence beginning at position 0, so the anchored splice is always in the output-exact regime of §IV. To preserve multi-turn coreference (e.g., “open a pull request *there*”), the sidecar carries a pruned sliding window: recent turns are kept verbatim while bulky tool payloads are replaced by short semantic surrogates. Given the resolved tool, arguments are generated in the main context over a *semantic compressed intermediate representation* (IR): the JSON schema is reduced to a dense, type-hinted function signature with truncated in-line descriptions, e.g. `create_repository(name: string, private?: boolean)`. At a median of 19 tokens the IR is far smaller than the full schema, so the main-context prefill avoids schema bloat while retaining enough semantics to bind arguments to the correct fields—and, on the routed cases, fills 100% of query-specified arguments correctly (§VII). A finite-state machine masks logits to the tool-name radix trie and to a JSON grammar (`nexus_fsm.cpp`).

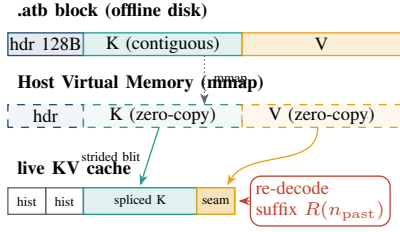


Fig. 3. Physical splice with memory mapping. The compiled block is mapped via zero-copy `mmap` into virtual memory. Contiguous F16 keys and values are then blitted into the live cache; the trailing suffix fraction $R(n_{\text{past}})$ is re-decoded to stitch the seam. Layouts are transposed for soft-capped architectures (§VI-B).

VI. SYSTEMS IMPLEMENTATION

A. Compiled Tool Blocks and Splice

Each schema is compiled offline into an Aeon Tool Block (`.atb`): a 128-byte header recording the RoPE anchor and scaling parameters, followed by page-aligned (2 MB) contiguous F16 key and value tensors (`aeon_tool_block.hpp`). The header’s RoPE parameters are validated against the runtime model before any splice, so a block compiled for a different scaling regime is rejected rather than silently corrupting attention. The splice writes K/V rows into the live KV cells at the runtime cursor (Fig. 3); a suffix of $\lceil S \cdot R(n_{\text{past}})/100 \rceil$ tokens is then invalidated and re-decoded to stitch the seam (`nexus_orchestrator.cpp`).

B. Transposed-V Splicing for Soft-Capped Attention

Attention-logit soft-capping (e.g., Gemma-2) disables FlashAttention, which forces the V cache into a transposed, token-innermost layout (`v_trans=true`). A fixed-stride row-major copy cannot address this layout. Nexus implements a layout-aware `splice_v_layer` routine (`nexus_kv_splicer.cpp`) that performs strided transposed copies directly in the physical cache under UMA, validates the live V strides before writing, and is fidelity-tested on the soft-capped Gemma-2-9B model. On non-UMA / discrete-GPU back-ends, where a transposed strided host-to-device transfer would be pathological, the splicer declines and the system falls back to prefill—a deliberate safety boundary, not a general path. Consequently, the physical splicing mechanism is fundamentally restricted to local execution of GGUF models that are flash-attention-compatible and non-soft-capped (such as Qwen2.5). Splicing cannot execute under local Gemma4 execution (due to soft-capping forcing a transposed layout that the splicer rejects, and loading errors under the pinned `llama.cpp` version) or cloud execution (due to the physical nature of cache transplantation requiring direct memory copies into local physical cache addresses). Soft-capped and cloud models must fall back entirely to text-prefill (Path B, yielding a $1.0\times$ speedup).

C. Cache-Line Hardening and Concurrency

The Python orchestration layer drives a single native `llama_context` whose internal decode is guarded by a C++

mutex. Fine-grained Python locking around individual decode steps races that mutex; Nexus instead serializes an entire turn (routing plus argument generation) under one coarse re-entrant lock, yielding deterministic, bit-stable outputs under concurrency. To remove cross-core false-sharing on the buddy allocator (`QuantizedBitmapAllocator`), the allocator is declared `alignas(128)` (matching `NEXUS_CACHE_LINE`) and its contended mutex is pinned to its own cache line, enforced at compile time:

```
static_assert(alignof(QuantizedBitmapAllocator)
              % 128 == 0);
```

We report this as an implemented hardening with a passing four-thread determinism test; we do not claim a throughput speedup, having measured none in isolation.

D. L0 Exact-Token Radix Arena

A zero-allocation radix cache (`nexus_seq_warm_cache`) manages a fixed pool of 32 sequence slots and reuses warm prefixes by exact-token longest-common-prefix matching, copying KV within a pre-reserved arena (`llama_kv_cache_seq_cp`) rather than allocating on the hot path.

VII. EVALUATION

All benchmarks were executed on an Apple M4 Max SoC (16-core CPU, 40-core GPU, 16-core Neural Engine) equipped with 64 GB Unified Memory and a 1 TB NVMe SSD, running macOS/Darwin 25.5.0 (arm64), at git 1ce4aa4 / `llama.cpp` cb2463bb. All measurements use Qwen2.5-14B-Instruct Q4_K_M with `nomic-embed-text-v1.5`. Artifacts are under `results/v2.0_canonical/`. Sample sizes are stated per result; they are small, and we treat the numbers as an envelope rather than a population estimate. To make that framing concrete rather than a disclaimer, every headline proportion carries a Wilson 95% interval and every TTFT median a bootstrap 95% interval (the deep-splice cells use $n=15$ trials); speedup intervals propagate the prefill and splice median CIs. We emphasize that all quantitative evaluation is restricted to this local, flash-attention-compatible, non-soft-capped configuration. Physical splicing is architecturally incompatible with remote cloud APIs (which accept only text token streams) and local soft-capped layouts (e.g., Gemma4) that disable FlashAttention and require transposed V-cache storage; these execute with zero system acceleration, falling back to standard text prefills.

A. Depth-Adaptive Splice: TTFT and Never-Regress

Table III reports median TTFT of the deep splice against a full text-prefill baseline, at increasing context depth, for the default curve ($K = 4$) and a flatter tuned curve ($K = 16$), with 15 trials per cell (`deep_splice_ttft.json`). Two facts hold across every cell: top-1 next-token agreement is preserved and the next-token $D_{\text{KL}}(\mathbf{p}_0 \parallel \mathbf{p}_{\text{splice}})$ is ≈ 0 —the never-regress guarantee. We stress that never-regress is a guarantee on output *fidelity* (top-1 agreement and $D_{\text{KL}} \approx 0$), not on latency: because the repaired suffix adds redecode work, measured TTFT can dip slightly below parity (e.g. $0.98\times$ at $n_{\text{past}} = 1024$,

TABLE III

DEEP-SPLICE TTFT VS. DEPTH ($n=15$ TRIALS/CELL). NEVER-REGRESS HOLDS EVERYWHERE (TOP-1 OK, $D_{KL}(\mathbf{p}_0 \parallel \mathbf{p}_{\text{splice}}) \approx 0$); SPEEDUP NARROWS TO PARITY AS RECOMPUTE ESCALATES. EQ. (2) REPRODUCES THE $R(\%)$ COLUMN EXACTLY (VERIFIED), SO THE RECOMPUTE SCHEDULE CAN BE AUDITED WITHOUT THE CODE. SPEEDUP INTERVALS PROPAGATE THE PER-CELL BOOTSTRAP 95% CIs OF THE PREFILL AND SPLICE MEDIANS (INDEPENDENT TIMING LOOPS). SOURCE: DEEP_SPLICE_TTFT.JSON.

n_{past}	$R(\%)$	prefill (ms)	splice (ms)	speedup [95% CI]	top-1/ D_{KL}
Default curve $K = 4$					
256	5.0	3278	2010	1.63 [1.62,1.67]	OK / ≈ 0
512	36.7	4672	3761	1.24 [1.23,1.25]	OK / ≈ 0
1024	100.0	7302	7425	0.98 [0.97,1.00]	OK / 0
2048	100.0	13092	13142	1.00 [0.99,1.00]	OK / 0
Tuned curve $K = 16$					
256	5.0	3327	1924	1.73 [1.72,1.75]	OK / ≈ 0
512	11.3	4747	3336	1.42 [1.42,1.43]	OK / ≈ 0
1024	24.0	7625	6532	1.17 [1.14,1.19]	OK / ≈ 0
2048	49.3	13468	12620	1.07 [1.01,1.08]	OK / ≈ 0

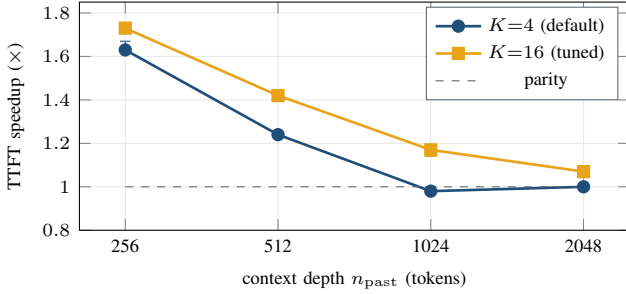


Fig. 4. TTFT speedup vs. depth for the two recompute curves (deep_splice_ttft.json). Higher K delays full recompute, sustaining speedup deeper at the cost of a larger repaired suffix; both converge to parity as $R(n_{\text{past}}) \rightarrow 100\%$.

$K = 4$) before converging to $1.0\times$; the fidelity guarantee is never violated even where the latency ratio is. The speedup is not flat: at moderate depth the splice saves $1.1\text{--}1.7\times$, and as $R(n_{\text{past}})$ escalates toward 100% the repaired splice converges to prefill parity ($\approx 1.0\times$). Flat TTFT and never-regress are in genuine tension, and K is the dial between them (Fig. 4).

B. Routing Accuracy and Registry Scale

Routing is evaluated over 100 GitHub-MCP queries as the registry scales from $N = 10$ to $N = 250$ tools (routing_accuracy_n250.json). End-to-end routing accuracy is nearly flat with scale: 92%, 90%, 89%, 89% at $N = 10, 50, 100, 250$ (Fig. 5); at $N = 250$ the routing accuracy carries a Wilson 95% CI of $[81.4, 93.7]$ ($n=100$), with SLB top-1 recall 74% $[64.6, 81.6]$ and top-3 recall 95% $[88.8, 97.8]$. The contrast with the concatenate-all-schemas oracle is the point: the oracle reaches 98% at $N = 10$ but *overflows the context window* at $N \geq 50$ and cannot answer at all, whereas Nexus keeps the main-context prompt small and continues to route. Because the oracle no longer fits the prompt at $N \geq 50$, a token-reduction ratio against it is undefined there; Nexus instead holds the main-context routing payload to a median

19-token IR (§V). Where a ratio is well defined—the sidecar’s hybrid path against a full-schema re-prefill—the main-context saving is $\approx 80\%$ (§VII). Because SLB search depends only on the query embedding and the registry, tool count does not inflate TTFT: the in-situ INT8 scan over 250 real tool vectors completes in $17.6\mu\text{s}$ (median, including the Python FFI boundary; routing_accuracy_n250.json), while the pure C++ SIMD dot-product scan over 250 unit-norm vectors runs in $8.25\mu\text{s}$ (median; slb_latency.json). Registering the tool name alongside its description under the `nomic` prefixes is what makes retrieval scale; without it, top-1 recall at $N = 250$ collapses. Table IV summarizes routing and micro-costs.

C. Argument Fidelity

Over the compressed textual IR (median 19 tokens, p99 32), the sidecar routes correctly on 86.7% (Wilson 95% CI $[70.3, 94.7]$) of the 30-case consistency set and, on the routed cases, fills 100% (95% CI $\geq 91.2\%$ over 40 specified arguments) of query-specified arguments correctly with a 100% JSON validity rate and no placeholder leakage (sidecar/accuracy.json); end-to-end argument accuracy, which charges routing errors as failures, is 80% (40/50 arguments, $[67.0, 88.8]$). The hybrid path reaches its first-argument token in 443.8 ms versus 737.3 ms for an oracle that re-prefills the full schema every turn—a $1.66\times$ reduction at an $\approx 80\%$ main-context token saving. The pruned sliding window of §V keeps recent-turn referents available for cross-turn arguments without re-admitting bulky schema text; we do not report a separate coreference accuracy, as no such measurement is in the committed v2.0 bundle.

D. Discussion

The evaluation supports a bounded thesis. Off-anchor KV splicing is fundamentally limited by RoPE phase drift, but the failure envelope is predictable and can be repaired by a deterministic depth-adaptive recompute that never regresses below a prefill. On this workload that buys a $1.1\text{--}1.7\times$ TTFT reduction at moderate depth, decaying to parity at deep context. Routing, argument generation, and coreference are deliberately

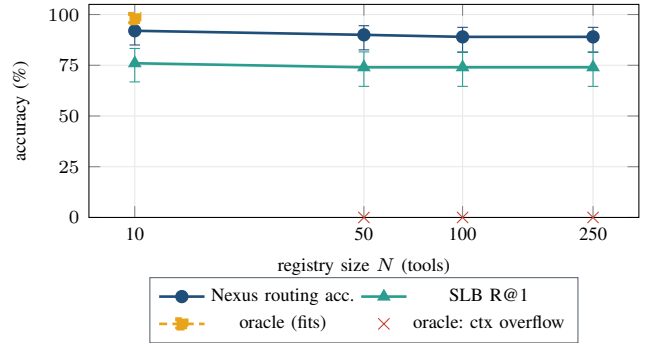


Fig. 5. Routing accuracy vs. registry size (routing_accuracy_n250.json). Nexus stays near 89% as N grows to 250; the concatenate-all oracle is accurate at $N = 10$ but overflows the context window at $N \geq 50$ and cannot route at all.

TABLE IV

ROUTING AND MICRO-COSTS AT $N=250$ TOOLS. PROPORTIONS CARRY WILSON 95% INTERVALS (BRACKETED); $\geq x$ DENOTES THE LOWER BOUND OF A 100% CELL. SOURCES: ROUTING_ACCURACY_N250.JSON, SLB_LATENCY.JSON, SIDECAR/ACCURACY.JSON, CALIBRATION.JSON, L0_RADIX.JSON.

Quantity	Value
Routing accuracy, $N=250$ ($n=100$)	89 % [81.4, 93.7]
SLB top-1 / top-3 recall, $N=250$	74 / 95 %
SLB search latency, in-situ incl. FFI ($N=250$)	17.6 μ s
SLB search latency, pure C++ SIMD scan	8.25 μ s
L0 radix copy latency (P50, 10 seeds)	3.04 μ s
L0 radix warm-hit rate (10 seeds)	69.5 %
Sidcar routing accuracy ($n=30$ cases)	86.7 % [70.3, 94.7]
Specified-arg accuracy (routed, $n=40$ args)	100 % (≥ 91.2)
End-to-end argument accuracy ($n=50$ args)	80 % [67.0, 88.8]
JSON validity rate ($n=30$)	100 % (≥ 88.6)
IR length, median / p99	19 / 32 tok
Cross-encoder gate τ (P20)	0.0136
Gate fire rate	20.3 %
Reference-free gate Spearman ρ	0.193

kept out of the splice path—a design forced by the off-anchor fidelity boundary—and run over retrieval and a compact textual IR at microsecond-scale overhead.

VIII. LIMITATIONS AND SCOPE

We state the boundaries of these claims explicitly. *Single tuple*. All quantitative numbers are from one host (Apple M4 Max, UMA) and one measured model tuple (Qwen2.5-14B-Instruct Q4_K_M) with one embedder and one llama.cpp build; the transposed-V splice mechanism is additionally validated on Gemma-2-9B, but generality beyond these is unproven. *Small n* . End-to-end arms use $n \leq 30$. Accuracy intervals are correspondingly wide and we do not report them as population estimates. *Deep splice is micro-validated*. The never-regress curve is measured at the fidelity-and-latency level (Table III); it is wired into the agent (enable_deep_splice) but not exercised as a multi-turn production path. *Transposed-V is UMA-only*. Soft-capped models splice under unified memory; discrete-GPU back-ends decline by design. *Retrieval ceiling*. Dense and cross-encoder recall have a workload-dependent ceiling; the gate improves the lowest-confidence decisions but does not remove hard confusions.

IX. CONCLUSION

Nexus reframes tool-schema acceleration from unconditional KV transplantation to retrieval-decoupled routing paired with a depth-adaptive repair of the schema splice. We characterized the RoPE-induced fidelity boundary in output-distribution terms, established the off-anchor boundary and the reference-free gating failure as well-supported negative results. We design the serving substrate to decouple semantic routing from the attention-level phase drift boundary, executing partial recomputes only where mathematically required to restore output fidelity. On a single-host 14B prototype this delivers a $1.1\text{--}1.7\times$ TTFT reduction at moderate depth with high argument validity. We view the measured envelope and the negative results as the durable

contribution, and we separate what generalizes from what does not: the *qualitative* boundaries—off-anchor relocation drifts, scattered recompute cannot cheaply repair it, and reference-free gating fails to predict it—are properties of RoPE and hold model-agnostically, whereas the *quantitative* envelope (the $P = 256$ threshold, the recompute-curve constants, and the specific divergence magnitudes) is calibrated to one model tuple and must be re-measured elsewhere.

REFERENCES

- [1] Anthropic, “Model context protocol specification,” <https://modelcontextprotocol.io>, 2024.
- [2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, 2017.
- [3] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, “RoFormer: Enhanced transformer with rotary position embedding,” *Neurocomputing*, vol. 568, 2024.
- [4] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with pagedattention,” in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.
- [5] L. Zheng, L. R. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, “Sglang: Efficient execution of structured language model programs,” in *Advances in Neural Information Processing Systems*, 2024.
- [6] T. Gan and Q. Sun, “RAG-MCP: Mitigating prompt bloat in LLM tool selection via retrieval-augmented generation,” *arXiv preprint arXiv:2505.03275*, 2025.
- [7] F. Sakizli, “TSCG: Deterministic tool-schema compilation for agentic LLM deployments,” *arXiv preprint arXiv:2605.04107*, 2026.
- [8] A. Krikorian, Y. Li, and J. J. Corso, “NTILC: Neural tool invocation via learned compression,” *arXiv preprint arXiv:2606.06566*, 2026.
- [9] R. Prabhu, A. Nayak, J. Mohan, R. Ramjee, and A. Panwar, “vattention: Dynamic memory management for serving llms without pagedattention,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2025.
- [10] J. Xu, R. Zhang, Y. Xiong, C. Guo, Z. Liu, Y. Zhou, W. Hu, H. Wu, C. Shao, Z. Wang, Y. Yuan, J. Zhao, M. Guo, and J. Leng, “ellm: Elastic memory management framework for efficient llm serving,” *arXiv preprint arXiv:2506.15155*, 2025.
- [11] Y. Liu, Z. Luo, H. Jin, Z. Wang, R. He, B. Wang, G. Chen, and J. Hu, “RedKnot: Efficient long-context LLM serving with head-aware KV reuse and SegPagedAttention,” *arXiv preprint arXiv:2606.06256*, 2026.
- [12] L. Stepanek, “Fail-closed lowering of resident KV claims onto LLM serving runtimes,” *arXiv preprint arXiv:2606.01387*, 2026.
- [13] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee, “Taming throughput-latency tradeoff in llm inference with sarathi-serve,” in *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
- [14] P. Patel, E. Choukse, C. Zhang, Í. Goiri, A. Shah, S. Maleki, and R. Bianchini, “Splitwise: Efficient generative llm inference using phase splitting,” in *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024.
- [15] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, “Distserve: Disaggregating prefill and decoding for goodput-driven llm serving,” in *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
- [16] A. Dongre, “Transactional kv caching for speculative decoding under paged kv memory,” *TechRxiv preprint*, 2026.
- [17] B. Peng, J. Quesnelle, H. Fan, and E. Shippole, “Yarn: Efficient context window extension of large language models,” in *International Conference on Learning Representations*, 2024.
- [18] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “Flashattention: Fast and memory-efficient exact attention with io-awareness,” in *Advances in Neural Information Processing Systems*, 2022.
- [19] T. Dao, “Flashattention-2: Faster attention with better parallelism and work partitioning,” in *International Conference on Learning Representations*, 2024.

- [20] B. T. Willard and R. Louf, “Efficient guided generation for large language models,” *arXiv preprint arXiv:2307.09702*, 2023.
- [21] L. Beurer-Kellner, M. Fischer, and M. Vechev, “Prompting is programming: A query language for large language models,” in *Proceedings of the 44th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2023.
- [22] Microsoft Guidance Team, “Guidance: A language for controlling large language models,” <https://github.com/guidance-project/guidance>, 2023.
- [23] M. Arslan, “Aeon: High-performance neuro-symbolic memory management for long-horizon llm agents,” *arXiv preprint arXiv:2601.15311*, 2026.